

El examen tiene una duración de 1:30 horas.

Ejercicio de Programación (6p)

Se desea implementar una aplicación web para almacenar los tiempos por vuelta en una carrera de F1. Solo se almacenarán los tiempos por vuelta de un solo piloto (Fernando Alonso). Esta aplicación web deberá tener las siguientes características:

- Funcionalidades:
 - Al entrar en la aplicación se mostrará el título “Las 33 primeras vueltas de Alonso”, un formulario para dar de alta un nuevo tiempo y la lista de tiempos añadidos.
 - Los datos de un tiempo son: el número de vuelta y el tiempo por vuelta en formato “00:00:00”.
 - La lista de tiempos debe mostrar el número de vuelta y el tiempo en una única línea.
 - Los datos de un nuevo tiempo no tienen porque añadirse en orden, es decir se puede añadir primero el tiempo de la vuelta 5 y después el de la vuelta 1. Sin embargo, en la lista de tiempos deben mostrarse en orden de vuelta, primero los tiempos de la vuelta 1, después los de la vuelta 2, etc...
 - Cuando un usuario complete los datos de un nuevo tiempo y pulse el botón de “Añadir”, se enviarán los datos del nuevo tiempo al servidor, que los guardará en la base de datos y este nuevo tiempo aparecerá en la lista de tiempos.
 - Cuando un usuario quiere añadir un nuevo tiempo, el servidor verificará que la vuelta que quiere añadir no existe. En caso de que ya haya un tiempo asignado a la vuelta se mostrará un error al usuario.
 - En la lista de tiempos al lado de cada tiempo aparecerá el botón “Borrar”. Al pulsar dicho botón, el tiempo será borrado de la base de datos y dejará de mostrarse en la lista de tiempos.
- Cuestiones de implementación:
 - Se deberá implementar una única aplicación con interfaz web con arquitectura MVC tradicional (generando los HTML en el servidor) y con una interfaz SPA. Es decir, como hemos hecho en la práctica de la fase 4.
 - La aplicación web tradicional estará en la raíz de la URL (<http://localhost/>) y la aplicación SPA estará en (<http://localhost/nano/>).
 - No es necesario usar ningún tipo de estilo CSS ni librería de componentes.
 - La forma de mostrar el error al usuario cuando se intenta crear un contacto con un nombre existente será diferente en la arquitectura web tradicional y en SPA.
 - En arquitectura web tradicional será una nueva página de error que muestre el mensaje “Vuelta ya asignada” y un botón o link para volver a la lista de tiempos.
 - En SPA será una alerta de JavaScript que se mostrará con el código: `alert("Vuelta ya asignada");`.
 - El código deberá estar en inglés. Lo único que puede estar en castellano son los textos que aparecen en el interfaz de usuario.

Se pide:

- **A) Implementar la aplicación web con Spring MVC, SpringData y JPA. (2p)**
 - No es necesario escribir el fichero pom.xml, se puede asumir que tiene todas las dependencias correspondientes.
 - No es necesario escribir el fichero application.properties, se puede asumir que está conectada a una base de datos externa correctamente configurada.
 - No es necesario escribir la clase Application.
 - Es necesario escribir TODOS los demás ficheros de la aplicación.
 - No es necesario incluir los imports en los ficheros Java.

- No es necesario implementar los getter y setter de las clases Java. Se pueden dejar indicados con un comentario.
- **B) API REST (0.5p)**
 - Se debe crear una API REST respetando los principios de diseño: formato de las URLs, uso de los códigos de estado, cabeceras, métodos, etc.
 - No debe haber lógica duplicada con la web MVC.
- **C) Implementar el frontend usando Angular (2p)**
 - Se puede asumir que se dispone de un proyecto Angular creado con “ng new” con todos los ficheros necesarios (index.html, angular.cli, package.json, tsconfig.json, app.module.ts, etc.).
 - Hay que escribir el resto de ficheros necesarios para implementar los componentes, servicios y configuración de rutas (en caso de que sean necesarias).
 - No es necesario incluir los imports en los ficheros TypeScript.
 - Se usará HTML plano en el template de los componentes (sin ninguna librería de componentes como ng-bootstrap o angular-material).
 - Se asumirá que el proxy está correctamente configurado y se pueden usar URLs relativas para acceder a la API REST del backend.
- **D) Empaquetar la aplicación en un contenedor Docker (1.5p)**
 - Escribe un fichero Dockerfile multistate que construya una imagen con la aplicación (tanto backend como frontend) empaquetada.
 - Crea un script (build.sh o build.ps1 o build.bat) de construcción y publicación de la imagen docker usando ese Dockerfile. El script deberá contener el comando necesario para construir la imagen Docker con el nombre daw/tiempos:1.0.0 y el comando de publicación en DockerHub (se asume que el sistema está logueado en DockerHub con los permisos necesarios para publicar la imagen).
 - Se asumirá que el código Spring está alojado en una carpeta “backend” y el código Angular está alojado en una carpeta “frontend” y el Dockerfile y el script están en la carpeta raíz.
 - Se asumirá que el sistema donde se ejecute el script sólo tiene Docker instalado. No dispone de Node.js ni Maven.
 - Se asumirá que en DockerHub existen las imágenes maven:3.11.0, node:16.0.0 y openjdk-jre:17.0.0